VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

A Mini Project Report on

"Property Management System"

Submitted in Partial Fulfillment of the Requirements for the V Semester of the Degree of

Bachelor of Engineering in

Artificial Intelligence & Data Science

By

Fizza N (1CR23AD036)

Sai Jathin Reddy (1CR23AD052)

Dhanush K (1CR23AD035)

Aditya R Bennur (1CR23AD007)

Under the Guidance of,

Ms. Bhavana Thomas

Assistant Professor, Dept. of AI&DS

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CMR INSTITUTE OF TECHNOLOGY

#132,AECS Layout, IT Park Road, Kundanahalli, Bangalore-560037

CMR INSTITUTE OF TECHNOLOGY

#132,AECS Layout, IT Park Road, Kundanahalli, Bangalore-560037

Department of Artificial Intelligence and Data Science

Certificate

This is to certify that the Project work entitled **"Property Management System"** has been carried out by **Fizza N (1CR23AD036), Sai Jathin Reddy (1CR23AD052), Dhanush K (1CR23AD035), Aditya R Bennur (1CR23AD007)** , bonafide student of CMR Institute of Technology, Bengaluru in partial fulfillment for the award of the Degree of Bachelor of Engineering in Artificial Intelligence and Data Science of the Visvesvaraya Technological University, Belagavi during the year 2025-2026.It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. This mini-project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said Degree.

Ms Bhavana Thomas
Asst. Professor, Dept. of AI& DS

Dr. Shanthi M.B
Professor & HoD, Dept. of AI& DS

External Viva

Name of the Examiners	**Signature with Date**
1. _____	_____
2. _____	_____

Declaration

We, students of the V semester of Artificial Intelligence and Data Science, CMR Institute of Technology, Bangalore, declare that the mini project work entitled **"Property Management System"** has been successfully completed under the guidance of **Ms Bhavana Thomas**, Assistant Professor, Department of Artificial Intelligence and Data Science, CMR Institute of Technology, Bengaluru. This project work is submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in Computer Science and Engineering during the academic year 2025-2026. The matter embodied in the project report has not been submitted previously by anybody for the award of any degree or diploma to any university.

Place: Bangalore

Date: 12-11-2025

Fizza N (1CR23AD036)

Sai Jathin Reddy (1CR23AD052)

Dhanush K(1CR23AD035)

Aditya R Bennur (1CR23AD007)

Abstract

The **Property Management System** is a digital platform designed to streamline and automate the management of rental and paying guest (PG) properties. The system addresses key challenges of traditional manual management, such as delayed rent collection, mismanaged tenant records, and lack of transparency between owners, managers, and tenants. It provides three dedicated panels—**Admin**, **Manager**, and **Tenant**—enabling efficient role-based operations.

Admins can monitor property subscriptions and performance analytics, managers can handle tenants, rent, and complaints, while tenants can make digital payments, raise issues, and view property details. The application integrates automated reminders, secure cloud storage, and a payment gateway to ensure real-time updates and secure transactions.

Developed using **React.js**, **Node.js**, and **MySQL/MongoDB**, the system ensures scalability, security, and cross-platform compatibility (Web, Android, and iOS). The solution ultimately enhances operational efficiency, transparency, and user satisfaction in property management through digital automation and data-driven insights.

Acknowledgement

We take this opportunity to express our sincere gratitude and respect to CMR Institute of Technology, Bengaluru for providing us a platform to pursue our studies and carry out the Entrepreneurial Project.

It gives us immense pleasure to express our deep sense of gratitude to **Dr. Sanjay Jain,** Principal, CMRIT, Bengaluru, for his constant encouragement.

We would like to extend my sincere gratitude to **Dr. Shanthi M.B**, HOD, Department of Artificial Intelligence and Data Science, CMRIT, Bengaluru, who has been a constant support and encouragement throughout the course of this project.

We would like to thank our guide **Ms. Bhavana Thomas**, Assistant Professor, Department of Artificial Intelligence and Data Science, for the valuable guidance throughout the tenure of the project work.

We also thank all the faculty members of the Department of Artificial Intelligence and Data Science who directly or indirectly encouraged us.

Finally, We thank our parents and friends for all the moral support they have given us during the completion of this work.

Contents

Certificate i

Declaration ii

Abstract iii

Acknowledgement

1 Introduction 1

1.1 Problem Statement 1

1.2 Objectives 1

1.3 Relevance of Problem 2

1.4 Software Development Tool 3

1.5 Chapter-wise Summary 3

1.5.1 Chapter 1: Introduction 3

1.5.2 Chapter 2: Literature Review 4

1.5.3 Chapter 3: System Analysis and Design 4

1.5.4 Chapter 4: Implementation 4

1.5.5 Chapter 5: Testing and Results 4

1.5.6 Chapter 6: Conclusion and Future Scope 5

1.6 Chapter 2: Literature Survey 5

1.6.1 Existing Property Management Solutions 5

1.6.2 Technologies Used in the Proposed System 5

1.6.3 Comparison with Existing Systems 6

1.6.4 Summary 6

2 Proposed Model **7**

2.1 System Architecture 7

2.2 Module Design 11

2.3 Use Case Design 14

3 Implementation **20**

3.1 Login Page Code 20

3.2 Sign-Up Page Code 23

3.3 Dashboard Page Code 27

3.4 Add New Property Page Code 30

4 Result and Discussion **42**

4.1 Results 42

4.2 Discussion 43

5 Screenshots **44**

6 Testing **48**

6.1 Types of Testing Performed 48

6.1.1 Unit Testing 48

6.1.2 Integration Testing 48

6.1.3 Functional Testing 48

6.1.4 UI/UX Testing 49

6.1.5 Performance Testing 49

7 Conclusion and Future Scope **50**

References **52**

Chapter 1

Introduction

1.1 Problem Statement

In traditional property and paying guest (PG) management systems, most operations are handled manually, leading to inefficiency and a lack of transparency among property owners, managers, and tenants. Rent collection is often delayed due to the absence of automated reminders and digital payment options. Tenant records are frequently mismanaged, and complaints are not tracked or resolved efficiently.

Additionally, there is no centralized system that connects all stakeholders and provides real-time monitoring of payments, subscriptions, and building details. This results in poor communication, data inconsistency, and administrative delays. Therefore, there is a need for a **digital Property Management System** that automates operations, enhances transparency, and simplifies property-related tasks for all users.

1.2 Objectives

The main objective of this project is to design and develop a **Property Management System** that streamlines the management of rental and paying guest (PG) properties through automation and digital integration. The system aims to provide a unified platform for admins, managers, and tenants to interact efficiently.

The specific objectives of the project are as follows:

- To develop a web and mobile-based application with separate login panels for Admin, Manager, and Tenant.
- To automate rent collection through a secure online payment gateway.
- To implement an automatic reminder system for rent payments and important tasks.

- To enable tenants to raise complaints and track their resolution digitally.
- To allow admins to monitor managers, property subscriptions, and performance analytics.
- To securely store tenant and property data using cloud-based databases.
- To generate reports and statistics for data-driven decision making.

By achieving these objectives, the proposed system will reduce manual effort, enhance transparency, and improve operational efficiency in property management.

1.3 Relevance of Problem

Property and Paying Guest (PG) management involves multiple tasks such as rent collection, tenant record keeping, complaint handling, and property maintenance. Traditionally, these processes are handled manually using registers or spreadsheets, which often leads to errors, data loss, and miscommunication among owners, managers, and tenants.

With the rapid growth of urban housing and rental demand, it has become essential to adopt a digital solution that ensures accuracy, transparency, and efficiency in managing multiple properties. A centralized **Property Management System** helps automate repetitive tasks, streamline communication, and improve financial tracking.

The relevance of this problem lies in its impact on both property owners and tenants. By implementing a digital platform, property owners can easily monitor rent payments, handle complaints efficiently, and access real-time data, while tenants can enjoy a transparent, hassle-free experience.

Thus, the development of this system addresses a growing need for modernization and automation in the property management sector, reducing manual workload and enhancing overall operational effectiveness.

1.4 Software Development Tool

The proposed **Property Management System** is developed using modern web technologies and tools that ensure scalability, performance, and cross-platform compatibility. The system follows a three-tier architecture consisting of the frontend, backend, and database layers.

The software development environment and tools used for this project are as follows:

- **Frontend:** React.js is used to design an interactive and responsive user interface for web applications. For mobile platforms, React Native or Flutter can be used to build Android and iOS applications.
- **Backend:** Node.js with Express.js framework is used to handle server-side logic, API communication, and user authentication.
- **Database:** MySQL or MongoDB is used to store and manage tenant, property, and payment-related information securely in the cloud.
- **Payment Integration:** Razorpay or Stripe API is integrated to facilitate secure online rent transactions.
- **Cloud Deployment:** The system is deployed on cloud platforms such as AWS or Firebase for high availability and data security.
- **IDE Used:** Vercel serves as the main development environment for coding, testing, and debugging.

These tools collectively enable efficient development, testing, and deployment of the system, ensuring a robust and user-friendly property management platform.

1.5 Chapter-wise Summary

1.5.1 Chapter 1: Introduction

This chapter provides an overview of the Property Management System (PMS), its significance, and the scope of the project. It discusses the challenges faced by property owners and tenants

in managing properties manually and highlights how an automated system can streamline tasks such as tenant management, rent tracking, maintenance requests, and reporting. The objectives and expected outcomes of the system are also outlined.

1.5.2 Chapter 2: Literature Review

This chapter reviews existing property management solutions and technologies. It covers traditional manual methods, standalone software, and modern web-based or mobile applications. The chapter identifies the gaps in current systems, such as lack of real-time updates, limited reporting features, or poor user interface, which the proposed system aims to address.

1.5.3 Chapter 3: System Analysis and Design

In this chapter, the requirements of the PMS are analyzed in detail. Functional and non-functional requirements are specified. System design aspects such as data flow diagrams (DFD), entity-relationship diagrams (ERD), and system architecture are discussed. This chapter ensures that the system design aligns with user needs and project objectives.

1.5.4 Chapter 4: Implementation

This chapter explains the technologies, tools, and programming languages used to develop the system. The actual coding, database setup, user interface design, and integration of various modules such as tenant management, property listing, maintenance tracking, and reporting are described. Screenshots of key modules can be included to illustrate the implementation.

1.5.5 Chapter 5: Testing and Results

This chapter details the testing methodologies used, including unit testing, integration testing, system testing, and user acceptance testing. Test cases and their outcomes are presented to demonstrate that the system meets all specified requirements. Any errors or issues identified and how they were resolved are also discussed.

1.5.6 Chapter 6: Conclusion and Future Scope

This chapter summarizes the achievements of the project and evaluates the system's performance. The advantages of the automated Property Management System over manual methods are highlighted. Additionally, suggestions for future enhancements, such as mobile app integration, AI-based predictive maintenance, or cloud deployment, are discussed to indicate how the system can be expanded or improved.

1.6 Chapter 2: Literature Survey

The literature survey focuses on the technologies, frameworks, and tools used in the development of the Property Management System, along with a review of existing solutions. The primary goal is to analyze current approaches and justify the choices made for this project.

1.6.1 Existing Property Management Solutions

Several property management systems are available, ranging from desktop applications to web-based platforms. Most traditional systems provide basic functionalities such as tenant tracking, rent management, and maintenance scheduling. However, these solutions often lack real-time updates, cross-platform accessibility, and an interactive user interface, which are essential for modern property management.

1.6.2 Technologies Used in the Proposed System

The Property Management App developed in this project is a web-based platform hosted on Vercel. The key technologies and tools used include:

1. **Next.js and React:** For building a responsive and interactive front-end user interface.
2. **Supabase:** As a backend service providing authentication, database management, and real-time data syncing.
3. **Vercel:** For deploying the web application with scalable and serverless hosting.

4. **JavaScript / TypeScript:** For application logic, API calls, and component development.
5. **Tailwind CSS:** For styling and ensuring a modern, clean user interface.
6. **Node.js / Express (optional):** For custom server-side logic or API endpoints if needed.
7. **Code Editor (VS Code or PyCharm):** For writing and managing project code.
8. **Web Browser:** For testing and interacting with the deployed app.

1.6.3 Comparison with Existing Systems

Compared to conventional property management software, this system offers:

- Real-time updates for tenant and property data.
- Cloud-based access, eliminating the need for local installations.
- User-friendly dashboard for property owners and tenants.
- Secure authentication and role-based access using Supabase.
- Easy deployment and maintenance through Vercel.

1.6.4 Summary

The literature survey demonstrates that while many property management systems exist, most fail to provide real-time, accessible, and user-friendly solutions. The proposed web-based app addresses these gaps by leveraging modern web technologies and cloud deployment for efficient property management.

Chapter 2

Proposed Model

2.1 System Architecture

1. Users Layer

[leftmargin=*, noitemsep]

- **Owners / Landlords**

  [noitemsep]

  - Add, update, and remove property listings.
  - Manage tenants and rental agreements.
  - View reports and analytics for properties.

- **Tenants**

  [noitemsep]

  - Search and filter properties.
  - Book or rent properties online.
  - Make rental payments and track payment history.
  - Raise complaints or maintenance requests.

- **Admin**

  [noitemsep]

  - Manage users (tenants, owners).
  - Oversee properties and rental activities.
  - Generate reports and analytics.

2. Frontend Layer

[leftmargin=*, noitemsep]

- Web application built using **React / Next.js**.
- Mobile responsive design for tablets and smartphones.
- Handles user interface, property browsing, booking forms, and dashboards.
- Communicates with backend via **REST API or GraphQL**.

3. Backend / Application Layer

[leftmargin=*, noitemsep]

- Developed with **Node.js / Express** (or Django/Spring).
- Implements all business logic:

  [noitemsep]

  - Property management
  - Booking and rental processing
  - Payment processing integration
  - Notification dispatch
  - User authentication & authorization (JWT/OAuth)
- Acts as a bridge between frontend and databases/services.

4. Database Layer

[leftmargin=*, noitemsep]

- **Relational Database (PostgreSQL / MySQL)** for structured data:

[noitemsep]

  - Users, roles, tenants, landlords
  - Property details
  - Bookings and rental agreements
  - Payments and invoices

- Optional **NoSQL (MongoDB)** for unstructured data like logs or chat messages.

5. File Storage Layer

[leftmargin=*, noitemsep]

- Stores property images, documents, and attachments.
- Uses cloud services like **AWS S3 / Firebase Storage**.
- Supports efficient retrieval and secure storage.

6. Payment Gateway

[leftmargin=*, noitemsep]

- Integrates with **Stripe / Razorpay / PayPal**.
- Handles rental transactions securely.
- Supports notifications of successful payments.

7. Notification Services

[leftmargin=*, noitemsep]

- Email via SMTP / SendGrid.

- SMS via Twilio or similar.
- Push notifications via Firebase Cloud Messaging.
- Used for:

  [noitemsep]

  - Booking confirmations
  - Payment receipts
  - Maintenance alerts

8. Hosting & Deployment

[leftmargin=*, noitemsep]

- **Frontend:** Vercel, Netlify, or cloud hosting.
- **Backend:** Cloud server (AWS EC2, DigitalOcean) or managed service.
- **Database:** Managed cloud DB (AWS RDS, Azure SQL).
- **Static assets:** Served via CDN for fast performance.

9. Data Flow Summary

[leftmargin=*, noitemsep]

1. User interacts with the **frontend**.
2. Frontend sends requests to the **backend API**.
3. Backend processes requests using:

   [noitemsep]

   - **Database** for structured data
   - **File storage** for images/documents

- **Payment gateway** for transactions

4. Backend triggers **notifications** for users as needed.

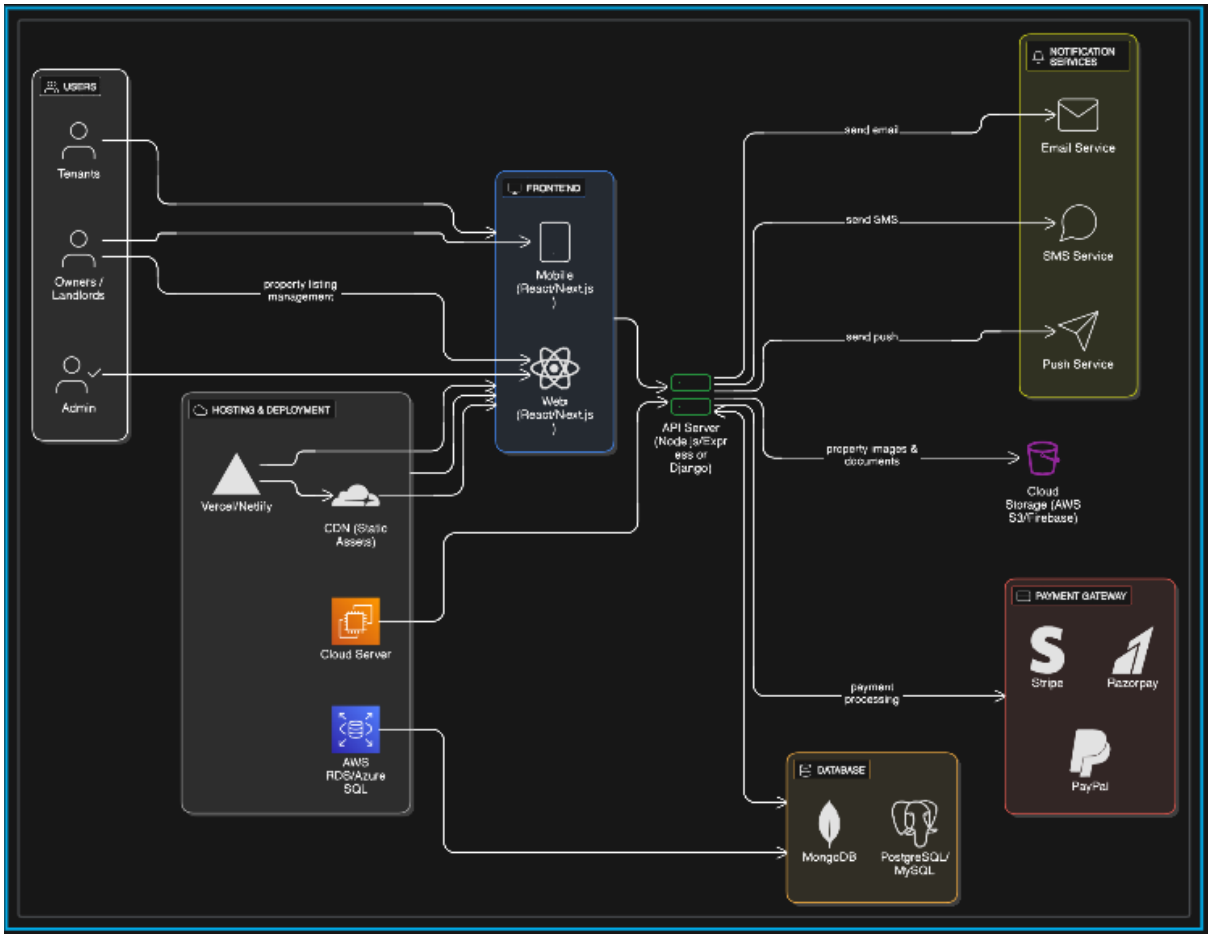

Figure 2.1: System Architecture

2.2 Module Design

Module Design

1. User Management Module

[leftmargin=*, noitemsep]

- Handles user registration, login, and profile management.
- Manages roles such as **Admin**, **Owner**, and **Tenant**.

- Provides authentication (JWT/OAuth) and access authorization.
- Stores user details, contact information, and permissions.

2. Property Management Module

[leftmargin=*, noitemsep]

- Allows owners to add, edit, and delete property listings.
- Stores property details including location, type, price, images, and amenities.
- Maintains property status (available, rented, booked).
- Enables tenants to search and filter properties.

3. Booking & Rental Management Module

[leftmargin=*, noitemsep]

- Manages booking requests, approvals, and rejections.
- Handles rental agreements, start/end dates, and lease terms.
- Tracks active and completed rentals.
- Generates invoices and rental summaries.

4. Payment Management Module

[leftmargin=*, noitemsep]

- Integrates with external payment gateways such as Razorpay or Stripe.
- Processes rent payments, security deposits, and refunds.
- Maintains payment history and generates receipts.
- Ensures secure transaction flow.

5. Complaint & Maintenance Module

[leftmargin=*, noitemsep]

- Allows tenants to raise complaints or maintenance requests.
- Assigns tasks to owners or maintenance personnel.
- Tracks complaint status such as *open*, *in-progress*, or *resolved*.
- Sends updates and notifications regarding request progress.

6. Notification Module

[leftmargin=*, noitemsep]

- Sends automated alerts via Email, SMS, or Push notifications.
- Notifies users about:

  [noitemsep]

  - Payment confirmations
  - Booking updates
  - Complaint status changes
  - Reminders and alerts
- Integrates services like SMTP, Twilio, or Firebase Cloud Messaging.

7. Admin Dashboard Module

[leftmargin=*, noitemsep]

- Enables admin to monitor system activity.
- Provides analytics on users, properties, bookings, and payments.
- Allows content management and dispute handling.
- Generates system-level reports.

8. File Storage Module

[leftmargin=*, noitemsep]

- Stores property images, documents, and rental agreements.
- Integrates with cloud services like AWS S3 or Firebase Storage.
- Ensures secure and efficient file retrieval.

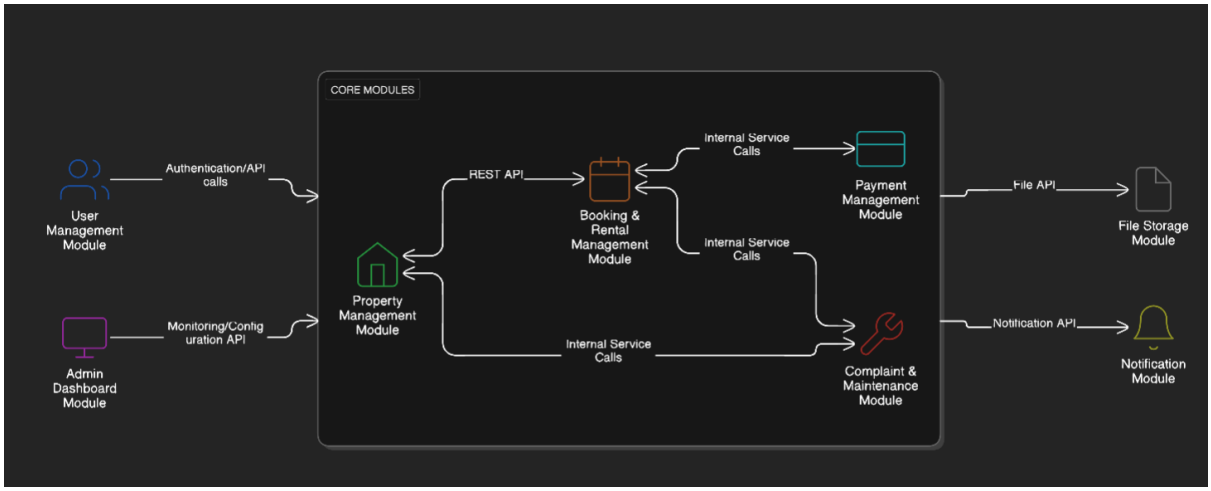

Figure 2.2: Module design

2.3 Use Case Design

Use Case Design

Actors

[leftmargin=*, noitemsep]

- **Admin** – Manages users, properties, reports, and system settings.
- **Owner / Landlord** – Manages properties, tenants, and rental activities.
- **Tenant** – Searches properties, makes bookings, payments, and raises complaints.

- **Payment Gateway** – Processes online transactions.
- **Notification Service** – Sends emails, SMS, and push notifications.

Use Case List

[leftmargin=*, noitemsep]

1. User Registration and Login
2. Manage User Profiles
3. Add / Edit / Delete Property
4. Search Properties
5. Book a Property
6. Manage Rental Agreement
7. Make Payment
8. View Payment History
9. Raise Complaint / Maintenance Request
10. Track Complaint Status
11. Generate Reports (Admin)
12. Receive Notifications

Use Case Descriptions

Use Case 1: User Login

Primary Actor: User (Admin / Owner / Tenant)

Precondition:

[noitemsep]

- User account must exist in the system.

Main Flow:

[noitemsep]

1. User enters email and password.
2. System validates credentials.
3. System grants access to the user dashboard.

Postcondition:

[noitemsep]

- User is successfully logged into the system.

Use Case 2: Add Property

Primary Actor: Owner

Precondition:

[noitemsep]

- Owner must be logged in.

Main Flow:

[noitemsep]

1. Owner selects “Add Property” option.
2. System displays property detail form.
3. Owner enters details such as location, price, photos, and amenities.

4. System saves property in the database.

Postcondition:

[noitemsep]

- Property becomes visible to tenants for searching.

Use Case 3: Book a Property

Primary Actor: Tenant

Precondition:

[noitemsep]

- Tenant must be logged in.
- Property must be available.

Main Flow:

[noitemsep]

1. Tenant selects a property.
2. System displays property details and booking options.
3. Tenant submits booking request.
4. Owner receives and approves the booking.
5. System confirms booking.

Postcondition:

[noitemsep]

- Booking details saved and tenant notified.

Use Case 4: Make Payment

Primary Actor: Tenant

Precondition:

[noitemsep]

- Tenant must be logged in.
- Booking must be approved.

Main Flow:

[noitemsep]

1. Tenant selects “Make Payment”.
2. System redirects to payment gateway.
3. Payment gateway processes the transaction.
4. System updates payment status.

Postcondition:

[noitemsep]

- Payment receipt generated and notification sent.

Use Case 5: Raise Complaint

Primary Actor: Tenant

Precondition:

[noitemsep]

- Tenant must be logged in.

Main Flow:

[noitemsep]

1. Tenant selects “Raise Complaint”.
2. System displays complaint form.
3. Tenant enters issue details.
4. System submits complaint to owner.

Postcondition:

[noitemsep]

- Complaint is stored and assigned for resolution.

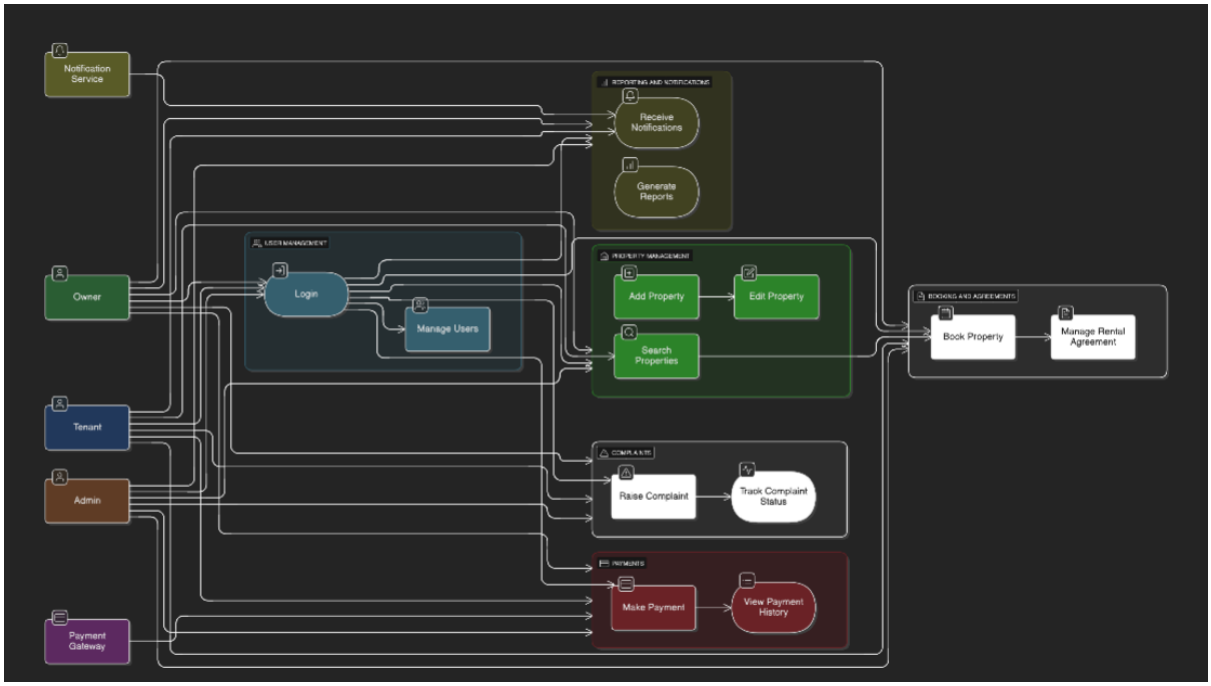

Figure 2.3: Module design

Chapter 3

Implementation

3.1 Login Page Code

```
"use client"

import type React from "react"

import { createClient } from "@/lib/supabase/client"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "
    @/components/ui/card"
import { Input } from "@/components/ui/input"
import { Label } from "@/components/ui/label"
import Link from "next/link"
import { useRouter } from "next/navigation"
import { useState } from "react"

export default function LoginPage() {
  const [email, setEmail] = useState("")
  const [password, setPassword] = useState("")
  const [error, setError] = useState<string | null>(null)
  const [isLoading, setIsLoading] = useState(false)
  const router = useRouter()

  const handleLogin = async (e: React.FormEvent) => {
    e.preventDefault()
    const supabase = createClient()
    setIsLoading(true)
    setError(null)

    try {
      const { error } = await supabase.auth.signInWithPassword({
        email,
```

```
      password,
    })
    if (error) throw error
    router.push("/dashboard")
  } catch (error: unknown) {
    setError(error instanceof Error ? error.message : "An error occurred
       ")
  } finally {
    setIsLoading(false)
  }
}

return (
  <div className="flex min-h-svh w-full items-center justify-center p-6
     md:p-10 bg-gradient-to-br from-background to-muted">
    <div className="w-full max-w-sm">
      <div className="flex flex-col gap-6">
        <div className="flex flex-col gap-2 text-center">
          <h1 className="text-3xl font-bold tracking-tight">SmartStay</h1
             >
          <p className="text-sm text-muted-foreground">Property
             Management System</p>
        </div>

        <Card className="border-0 shadow-lg">
          <CardHeader>
            <CardTitle className="text-2xl">Login</CardTitle>
            <CardDescription>Enter your credentials to access your
               properties</CardDescription>
          </CardHeader>
          <CardContent>
            <form onSubmit={handleLogin}>
              <div className="flex flex-col gap-6">
                <div className="grid gap-2">
                  <Label htmlFor="email">Email</Label>
                  <Input
                    id="email"
                    type="email"
                    placeholder="you@example.com"
```

```
                    required
                    value={email}
                    onChange={(e) => setEmail(e.target.value)}
                    disabled={isLoading}
                  />
                </div>
                <div className="grid gap-2">
                  <div className="flex items-center justify-between">
                    <Label htmlFor="password">Password</Label>
                  </div>
                  <Input
                    id="password"
                    type="password"
                    required
                    value={password}
                    onChange={(e) => setPassword(e.target.value)}
                    disabled={isLoading}
                  />
                </div>
                {error && <p className="text-sm text-destructive">{error
                    }</p>}
                <Button type="submit" className="w-full" disabled={
                    isLoading}>
                  {isLoading ? "Logging in..." : "Login"}
                </Button>
              </div>
              <div className="mt-4 text-center text-sm">
                {"Don't have an account? "}
                <Link href="/auth/sign-up" className="font-medium text-
                    primary hover:underline">
                  Sign up
                </Link>
              </div>
            </form>
          </CardContent>
        </Card>
      </div>
    </div>
  </div>
```

```
  )
}
```

3.2 Sign-Up Page Code

```
"use client"

import type React from "react"

import { createClient } from "@/lib/supabase/client"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "
    @/components/ui/card"
import { Input } from "@/components/ui/input"
import { Label } from "@/components/ui/label"
import Link from "next/link"
import { useRouter } from "next/navigation"
import { useState } from "react"

export default function SignUpPage() {
  const [email, setEmail] = useState("")
  const [password, setPassword] = useState("")
  const [confirmPassword, setConfirmPassword] = useState("")
  const [fullName, setFullName] = useState("")
  const [error, setError] = useState<string | null>(null)
  const [isLoading, setIsLoading] = useState(false)
  const router = useRouter()

  const handleSignUp = async (e: React.FormEvent) => {
    e.preventDefault()
    const supabase = createClient()
    setIsLoading(true)
    setError(null)

    if (password !== confirmPassword) {
      setError("Passwords do not match")
      setIsLoading(false)
```

```
    return
  }

  try {
    const { error } = await supabase.auth.signUp({
      email,
      password,
      options: {
        emailRedirectTo:
          process.env.NEXT_PUBLIC_DEV_SUPABASE_REDIRECT_URL || `${window.
              location.origin}/auth/callback`,
        data: {
          full_name: fullName,
        },
      },
    })
    if (error) throw error
    router.push("/auth/sign-up-success")
  } catch (error: unknown) {
    setError(error instanceof Error ? error.message : "An error occurred
        ")
  } finally {
    setIsLoading(false)
  }
}

return (
  <div className="flex min-h-svh w-full items-center justify-center p-6
      md:p-10 bg-gradient-to-br from-background to-muted">
    <div className="w-full max-w-sm">
      <div className="flex flex-col gap-6">
        <div className="flex flex-col gap-2 text-center">
          <h1 className="text-3xl font-bold tracking-tight">SmartStay</h1
              >
          <p className="text-sm text-muted-foreground">Create your
              account</p>
        </div>

        <Card className="border-0 shadow-lg">
```

```
            <CardHeader>
              <CardTitle className="text-2xl">Sign up</CardTitle>
              <CardDescription>Start managing your properties today</
                 CardDescription>
            </CardHeader>
            <CardContent>
              <form onSubmit={handleSignUp}>
                <div className="flex flex-col gap-6">
                  <div className="grid gap-2">
                    <Label htmlFor="fullName">Full Name</Label>
                    <Input
                      id="fullName"
                      type="text"
                      placeholder="John Doe"
                      required
                      value={fullName}
                      onChange={(e) => setFullName(e.target.value)}
                      disabled={isLoading}
                    />
                  </div>
                  <div className="grid gap-2">
                    <Label htmlFor="email">Email</Label>
                    <Input
                      id="email"
                      type="email"
                      placeholder="you@example.com"
                      required
                      value={email}
                      onChange={(e) => setEmail(e.target.value)}
                      disabled={isLoading}
                    />
                  </div>
                  <div className="grid gap-2">
                    <Label htmlFor="password">Password</Label>
                    <Input
                      id="password"
                      type="password"
                      required
                      value={password}
```

```
                    onChange={(e) => setPassword(e.target.value)}
                    disabled={isLoading}
                  />
                </div>
                <div className="grid gap-2">
                  <Label htmlFor="confirmPassword">Confirm Password</
                     Label>
                  <Input
                    id="confirmPassword"
                    type="password"
                    required
                    value={confirmPassword}
                    onChange={(e) => setConfirmPassword(e.target.value)}
                    disabled={isLoading}
                  />
                </div>
                {error && <p className="text-sm text-destructive">{error
                   }</p>}
                <Button type="submit" className="w-full" disabled={
                   isLoading}>
                  {isLoading ? "Creating account..." : "Sign up"}
                </Button>
              </div>
              <div className="mt-4 text-center text-sm">
                Already have an account?{" "}
                <Link href="/auth/login" className="font-medium text-
                   primary hover:underline">
                  Login
                </Link>
              </div>
            </form>
          </CardContent>
        </Card>
      </div>
    </div>
  </div>
  )
}
```

3.3 Dashboard Page Code

```
import { redirect } from "next/navigation"
import { createClient } from "@/lib/supabase/server"
import { Navbar } from "@/components/navbar"
import { Sidebar } from "@/components/sidebar"
import { StatsOverview } from "@/components/stats-overview"
import { PropertyCard } from "@/components/property-card"
import { Button } from "@/components/ui/button"
import Link from "next/link"

export default async function DashboardPage() {
  const supabase = await createClient()

  const {
    data: { user },
    error: userError,
  } = await supabase.auth.getUser()
  if (userError || !user) {
    redirect("/auth/login")
  }

  // Fetch properties
  const { data: properties = [] } = await supabase
    .from("properties")
    .select("*")
    .eq("owner_id", user.id)

  // Fetch tenants count
  const { data: tenants = [] } = await supabase
    .from("tenants")
    .select("id")
    .in(
      "property_id",
      properties.map((p: any) => p.id),
    )

  // Fetch active leases
  const { data: leases = [] } = await supabase
```

```
  .from("leases")
  .select("*")
  .eq("is_active", true)
  .in(
    "property_id",
    properties.map((p: any) => p.id),
  )

// Fetch pending payments
const { data: payments = [] } = await supabase
  .from("payments")
  .select("*")
  .eq("status", "pending")
  .in(
    "property_id",
    properties.map((p: any) => p.id),
  )

// Fetch maintenance requests
const { data: maintenance = [] } = await supabase
  .from("maintenance_requests")
  .select("*")
  .neq("status", "completed")
  .in(
    "property_id",
    properties.map((p: any) => p.id),
  )

return (
  <div className="flex flex-col h-screen bg-background">
    <Navbar />
    <div className="flex flex-1 overflow-hidden">
      <Sidebar />
      <main className="flex-1 overflow-y-auto">
        <div className="max-w-7xl mx-auto p-6 space-y-8">

          <div>
            <h1 className="text-3xl font-bold mb-2">Welcome back!</h1>
            <p className="text-muted-foreground">
```

```
            Here's an overview of your property management
          </p>
        </div>

        <StatsOverview
          totalProperties={properties.length}
          activeLeases={leases.length}
          totalTenants={tenants.length}
          pendingPayments={payments.length}
          maintenanceRequests={maintenance.length}
        />

        <div>
          <div className="flex items-center justify-between mb-4">
            <h2 className="text-2xl font-bold">Your Properties</h2>
            <Button asChild>
              <Link href="/properties/new">Add Property</Link>
            </Button>
          </div>

          {properties.length > 0 ? (
            <div className="grid grid-cols-1 md:grid-cols-2
                lg:grid-cols-3 gap-6">
              {properties.map((property: any) => (
                <PropertyCard
                  key={property.id}
                  id={property.id}
                  name={property.name}
                  address={property.address}
                  city={property.city}
                  state={property.state}
                  bedrooms={property.bedrooms}
                  bathrooms={property.bathrooms}
                  type={property.property_type}
                />
              ))}
            </div>
          ) : (
            <div className="text-center py-12 border border-dashed
```

```
                    border-border rounded-lg">
                  <h3 className="text-lg font-medium mb-2">No properties
                      yet</h3>
                  <p className="text-muted-foreground mb-4">
                    Get started by adding your first property
                  </p>
                  <Button asChild>
                    <Link href="/properties/new">
                      Add Your First Property
                    </Link>
                  </Button>
                </div>
              )}
            </div>

          </div>
        </main>
      </div>
    </div>
  )
}
```

listings xcolor

3.4 Add New Property Page Code

```
"use client"

import type React from "react"

import { useState } from "react"
import { useRouter } from "next/navigation"
import { createClient } from "@/lib/supabase/client"
import { Navbar } from "@/components/navbar"
import { Sidebar } from "@/components/sidebar"
import { Button } from "@/components/ui/button"
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/
    card"
```

```
import { Input } from "@/components/ui/input"
import { Label } from "@/components/ui/label"
import { Textarea } from "@/components/ui/textarea"
import Link from "next/link"

export default function NewPropertyPage() {
  const router = useRouter()
  const [isLoading, setIsLoading] = useState(false)
  const [error, setError] = useState<string | null>(null)

  const [formData, setFormData] = useState({
    name: "",
    address: "",
    city: "",
    state: "",
    zip_code: "",
    property_type: "apartment",
    bedrooms: "",
    bathrooms: "",
    square_feet: "",
    purchase_price: "",
    purchase_date: "",
    description: "",
  })

  const handleChange = (
    e: React.ChangeEvent<
      HTMLInputElement | HTMLTextAreaElement | HTMLSelectElement
    >
  ) => {
    const { name, value } = e.target
    setFormData((prev) => ({ ...prev, [name]: value }))
  }

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault()
    const supabase = createClient()
    setIsLoading(true)
    setError(null)
```

```
try {
  const {
    data: { user },
  } = await supabase.auth.getUser()
  if (!user) throw new Error("Not authenticated")

  const { error: insertError } = await supabase
    .from("properties")
    .insert({
      owner_id: user.id,
      name: formData.name,
      address: formData.address,
      city: formData.city,
      state: formData.state,
      zip_code: formData.zip_code,
      property_type: formData.property_type,
      bedrooms: formData.bedrooms
        ? Number.parseInt(formData.bedrooms)
        : null,
      bathrooms: formData.bathrooms
        ? Number.parseFloat(formData.bathrooms)
        : null,
      square_feet: formData.square_feet
        ? Number.parseInt(formData.square_feet)
        : null,
      purchase_price: formData.purchase_price
        ? Number.parseFloat(formData.purchase_price)
        : null,
      purchase_date: formData.purchase_date || null,
      description: formData.description,
    })

  if (insertError) throw insertError
  router.push("/properties")
} catch (err: unknown) {
  setError(
    err instanceof Error ? err.message : "An error occurred"
  )
```

```
    } finally {
      setIsLoading(false)
    }
  }

  return (
    <div className="flex flex-col h-screen bg-background">
      <Navbar />
      <div className="flex flex-1 overflow-hidden">
        <Sidebar />
        <main className="flex-1 overflow-y-auto">
          <div className="max-w-2xl mx-auto p-6 space-y-6">
            <div>
              <h1 className="text-3xl font-bold">Add New Property</h1>
              <p className="text-muted-foreground mt-1">
                Enter the details for your new property
              </p>
            </div>

            <Card>
              <CardHeader>
                <CardTitle>Property Information</CardTitle>
              </CardHeader>
              <CardContent>
                <form onSubmit={handleSubmit} className="space-y-6">
                  <div className="grid grid-cols-2 gap-4">
                    <div className="col-span-2">
                      <Label htmlFor="name">Property Name</Label>
                      <Input
                        id="name"
                        name="name"
                        placeholder="e.g., Sunny Apartment Complex"
                        value={formData.name}
                        onChange={handleChange}
                        required
                        disabled={isLoading}
                      />
                    </div>
```

```
                <div className="col-span-2">
                  <Label htmlFor="address">Street Address</Label>
                  <Input
                    id="address"
                    name="address"
                    placeholder="123 Main St"
                    value={formData.address}
                    onChange={handleChange}
                    required
                    disabled={isLoading}
                  />
                </div>

                <div>
                  <Label htmlFor="city">City</Label>
                  <Input
                    id="city"
                    name="city"
                    placeholder="New York"
                    value={formData.city}
                    onChange={handleChange}
                    required
                    disabled={isLoading}
                  />
                </div>

                <div>
                  <Label htmlFor="state">State</Label>
                  <Input
                    id="state"
                    name="state"
                    placeholder="NY"
                    value={formData.state}
                    onChange={handleChange}
                    required
                    disabled={isLoading}
                  />
                </div>
```

```
                    <div>
                      <Label htmlFor="zip_code">Zip Code</Label>
                      <Input
                        id="zip_code"
                        name="zip_code"
                        placeholder="10001"
                        value={formData.zip_code}
                        onChange={handleChange}
                        required
                        disabled={isLoading}
                      />
                    </div>

                    <div>
                      <Label htmlFor="property_type">Property Type</Label>
                      <select
                        id="property_type"
                        name="property_type"
                        value={formData.property_type}
                        onChange={handleChange}
                        className="w-full px-3 py-2 border border-input
                        bg-background text-foreground rounded-md
                        focus:outline-none focus:ring-2 focus:ring-ring"
                        disabled={isLoading}
                      >
                        <option value="apartment">Apartment</option>
                        <option value="house">House</option>
                        <option value="condo">Condo</option>
                        <option value="townhouse">Townhouse</option>
                        <option value="commercial">Commercial</option>
                      </select>
                    </div>

                    <div>
                      <Label htmlFor="bedrooms">Bedrooms</Label>
                      <Input
                        id="bedrooms"
                        name="bedrooms"
                        type="number"
```

```
                    placeholder="2"
                    value={formData.bedrooms}
                    onChange={handleChange}
                    disabled={isLoading}
                  />
                </div>

                <div>
                  <Label htmlFor="bathrooms">Bathrooms</Label>
                  <Input
                    id="bathrooms"
                    name="bathrooms"
                    type="number"
                    step="0.5"
                    placeholder="1.5"
                    value={formData.bathrooms}
                    onChange={handleChange}
                    disabled={isLoading}
                  />
                </div>

                <div>
                  <Label htmlFor="square_feet">Square Feet</Label>
                  <Input
                    id="square_feet"
                    name="square_feet"
                    type="number"
                    placeholder="1500"
                    value={formData.square_feet}
                    onChange={handleChange}
                    disabled={isLoading}
                  />
                </div>

                <div>
                  <Label htmlFor="purchase_price">Purchase Price</Label
                      >
                  <Input
                    id="purchase_price"
```

```
            name="purchase_price"
            type="number"
            step="0.01"
            placeholder="250000"
            value={formData.purchase_price}
            onChange={handleChange}
            disabled={isLoading}
          />
        </div>

        <div>
          <Label htmlFor="purchase_date">Purchase Date</Label>
          <Input
            id="purchase_date"
            name="purchase_date"
            type="date"
            value={formData.purchase_date}
            onChange={handleChange}
            disabled={isLoading}
          />
        </div>

        <div className="col-span-2">
          <Label htmlFor="description">Description</Label>
          <Textarea
            id="description"
            name="description"
            placeholder="Add any additional details..."
            value={formData.description}
            onChange={handleChange}
            rows={4}
            disabled={isLoading}
          />
        </div>
      </div>

      {error && (
        <p className="text-sm text-destructive">{error}</p>
      )}
```

```
                  <div className="flex gap-4">
                    <Button type="submit" disabled={isLoading}>
                      {isLoading ? "Creating..." : "Create Property"}
                    </Button>
                    <Button type="button" variant="outline" asChild>
                      <Link href="/properties">Cancel</Link>
                    </Button>
                  </div>
                </form>
              </CardContent>
            </Card>
          </div>
        </main>
      </div>
    </div>
  )
}
```

PropertyCard Component

Listing 3.1: React/Next.js PropertyCard Component

```
"use client"

import Link from "next/link"
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/
    card"

interface PropertyCardProps {
  id: string
  name: string
  address: string
  city: string
  state: string
  bedrooms: number | null
  bathrooms: number | null
  type: string
```

```
}

export function PropertyCard({ id, name, address, city, state, bedrooms,
    bathrooms, type }: PropertyCardProps) {
  return (
    <Link href={`/properties/${id}`}>
      <Card className="hover:shadow-lg transition-shadow cursor-pointer h-
          full">
        <CardHeader>
          <CardTitle className="text-lg">{name}</CardTitle>
        </CardHeader>
        <CardContent className="space-y-3">
          <div>
            <p className="text-sm text-muted-foreground">{address}</p>
            <p className="text-sm text-muted-foreground">{city}, {state}</p
                >
          </div>

          <div className="flex gap-4 text-sm">
            {bedrooms !== null && (
              <div>
                <span className="font-semibold">{bedrooms}</span>
                <span className="text-muted-foreground ml-1">Bed</span>
              </div>
            )}
            {bathrooms !== null && (
              <div>
                <span className="font-semibold">{bathrooms}</span>
                <span className="text-muted-foreground ml-1">Bath</span>
              </div>
            )}
            <div className="ml-auto">
              <span className="inline-block px-2 py-1 bg-secondary text-
                  secondary-foreground text-xs rounded">
                {type}
              </span>
            </div>
          </div>
        </CardContent>
```

```
      </Card>
    </Link>
  )
}
```

Supabase Client

Listing 3.2: Supabase Client Initialization

```
import { createBrowserClient } from "@supabase/ssr"

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

Supabase Server Client

Listing 3.3: Supabase Server Client Initialization

```
import { createServerClient } from "@supabase/ssr"
import { cookies } from "next/headers"

export async function createClient() {
  const cookieStore = await cookies()

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll()
        },
        setAll(cookiesToSet) {
```

```
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            )
          } catch {
            // The "setAll" method was called from a Server Component.
            // This can be ignored if you have middleware refreshing
            // user sessions.
          }
        },
      },
    }
  )
}
```

Chapter 4

Result and Discussion

4.1 Results

The Property Management System was successfully designed and implemented using Next.js, React, TypeScript, Tailwind CSS, and Supabase as the backend service. The system provides a secure, responsive, and user–friendly interface for property owners to manage their real–estate portfolio.

The major outcomes of the project are summarized below:

- **User Authentication:** The system integrates Supabase Authentication, allowing secure login and preventing unauthorized access to property data.
- **Property Creation Module:** Users can add new properties using a structured form that collects details such as address, type, bedrooms, bathrooms, size, purchase price, and description. Validation and error–handling have been implemented.
- **Property Listing Module:** All properties belonging to a logged–in owner are displayed in a responsive grid using reusable UI components. Each property card provides quick access to details.
- **Property Detail View:** The system displays complete property information, along with counts of tenants, leases, and pending maintenance requests. Conditional rendering ensures clean and dynamic UI.
- **Database Integration:** All CRUD operations communicate with Supabase tables such as `properties`, `tenants`, `leases`, and `maintenance_requests`. The interaction was tested successfully with real data.
- **Navigation and UI Components:** A reusable `Navbar`, `Sidebar`, and `PropertyCard` component improve maintainability and scalability of the system.

Overall, the system performs all expected core functionalities and validates the feasibility of implementing a full–stack real–estate management solution using modern web technologies.

4.2 Discussion

The development of the Property Management System demonstrated the effectiveness of combining modern frontend technologies with a cloud–based backend service. The use of Next.js significantly improved routing performance and enabled the creation of both server–side and client–side rendered components. This hybrid model ensured fast page loads and secure data fetching.

The integration with Supabase played a critical role in simplifying backend development. Features such as authentication, role–based access, and real–time database operations were implemented without the need to manually build an API or authentication layer. This allowed the project to focus more on business logic and UI/UX.

During the design phase, reusable components such as the `PropertyCard`, `Navbar`, and `Sidebar` proved to be highly beneficial. They reduced code repetition and maintained consistency throughout the interface. The modular architecture also makes the application easier to extend in the future—for example, adding analytics dashboards, automated reminders, or rent tracking.

User experience was also a key aspect of the project. Form validation, conditional rendering, and responsive layout ensured that the application behaves correctly across different device sizes and input scenarios.

However, some limitations were observed. Since the project relies heavily on Supabase, the performance and availability of the platform directly affect the application. Additionally, features such as role management, advanced reporting, and offline support could be improved in future iterations.

Overall, the project successfully validates the use of serverless backend services with a modern React framework to build scalable property management applications.

Chapter 5

Screenshots

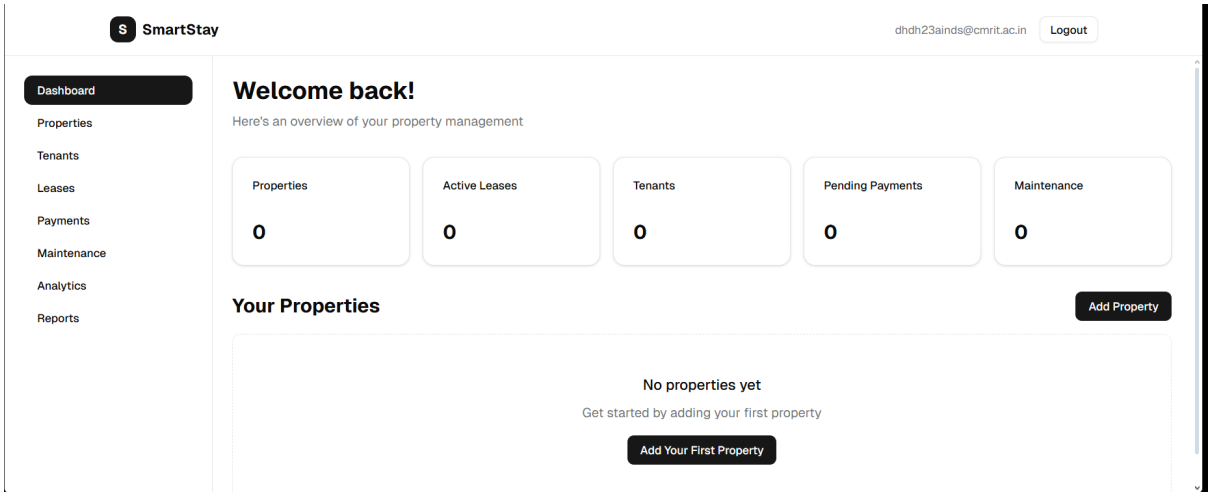

Figure 5.1: Dashboard

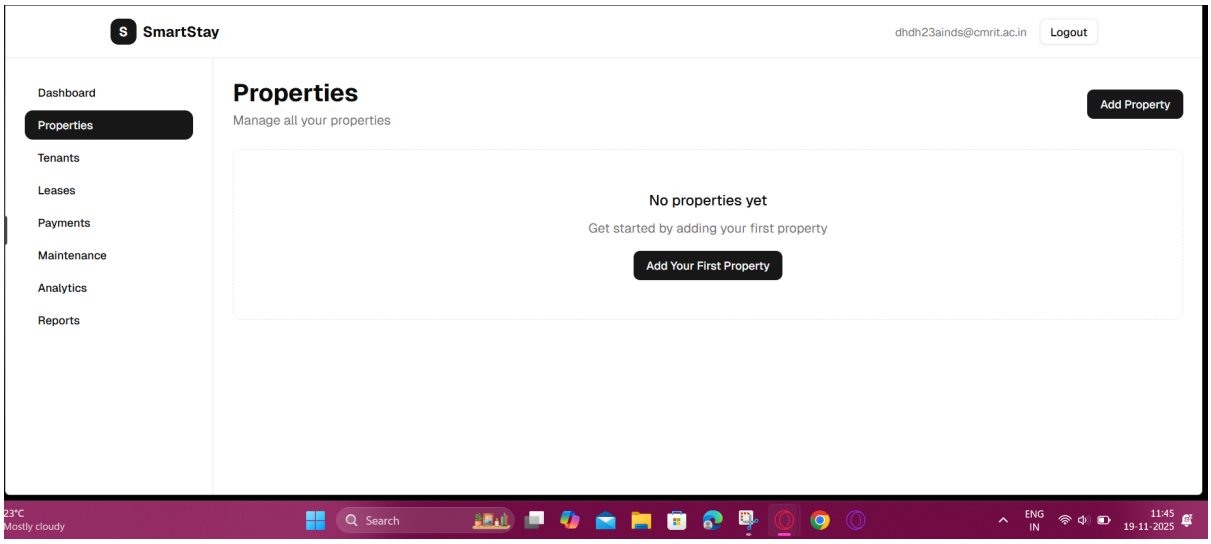

Figure 5.2: Property page

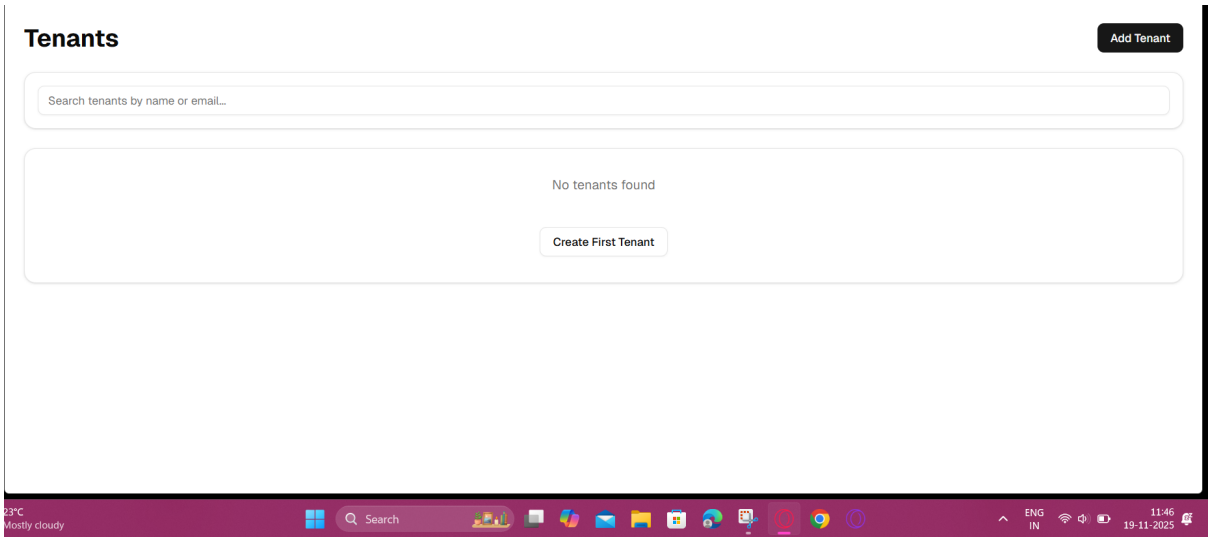

Figure 5.3: Tenants page

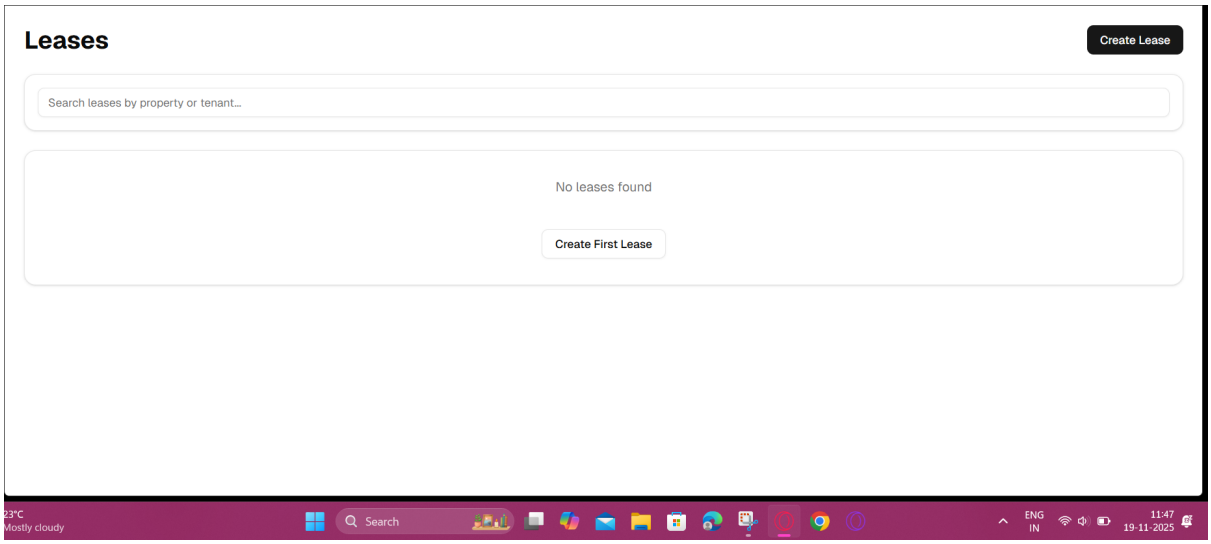

Figure 5.4: Lease Page

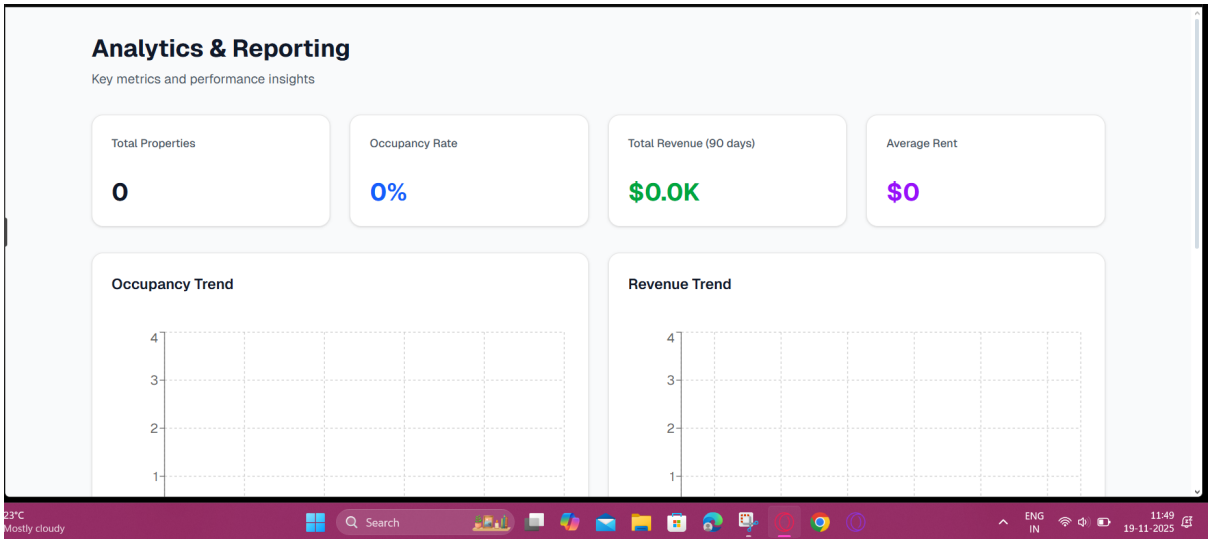

Figure 5.5: Analytics Reporting Page

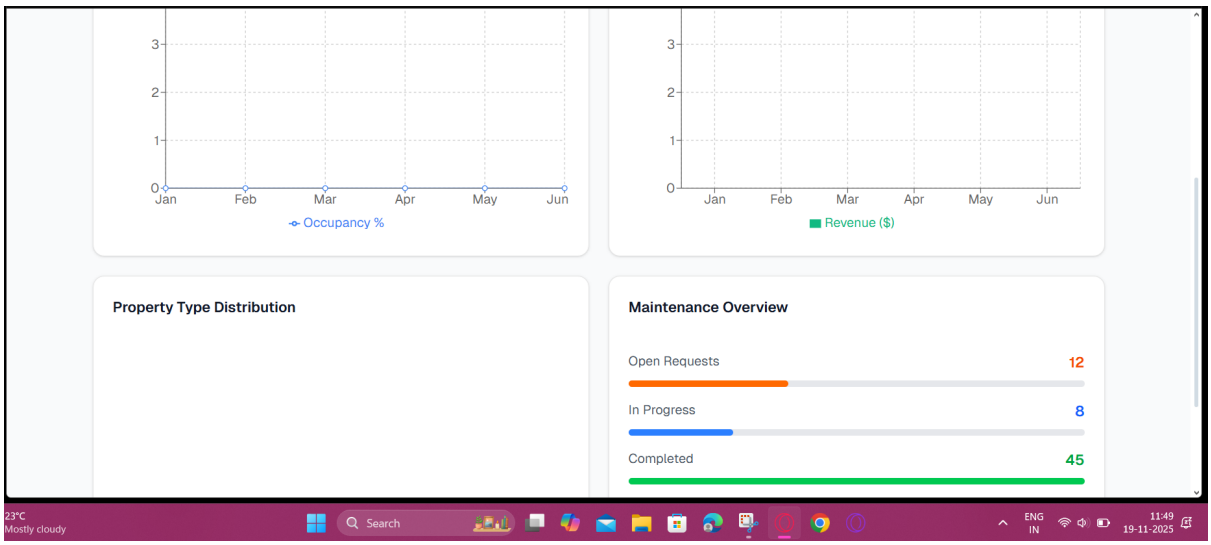

Figure 5.6: Analytics Reporting Page

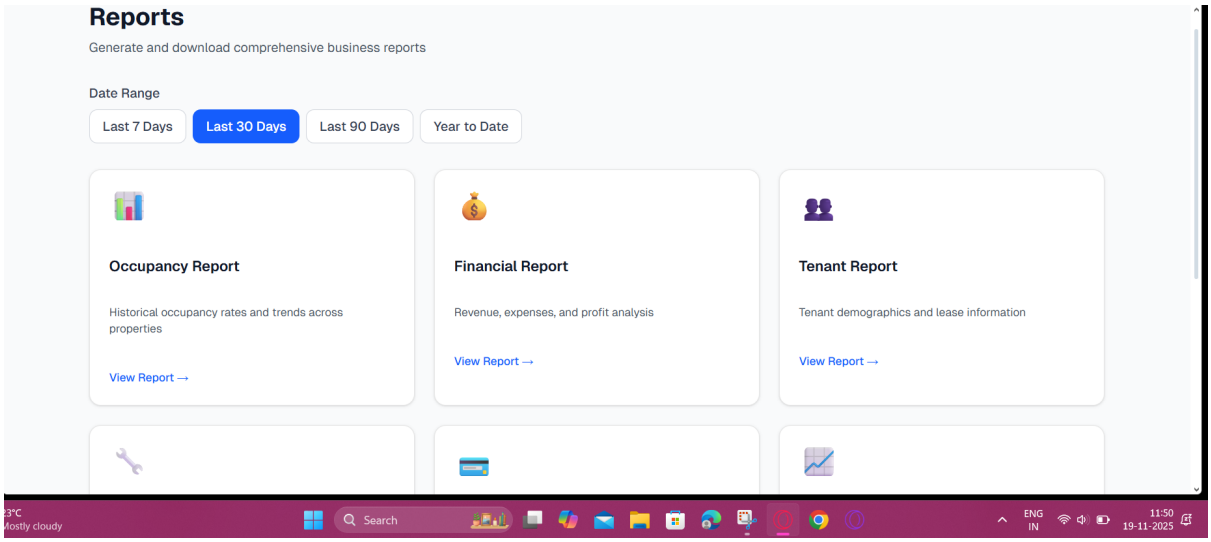

Figure 5.7: Reports Page

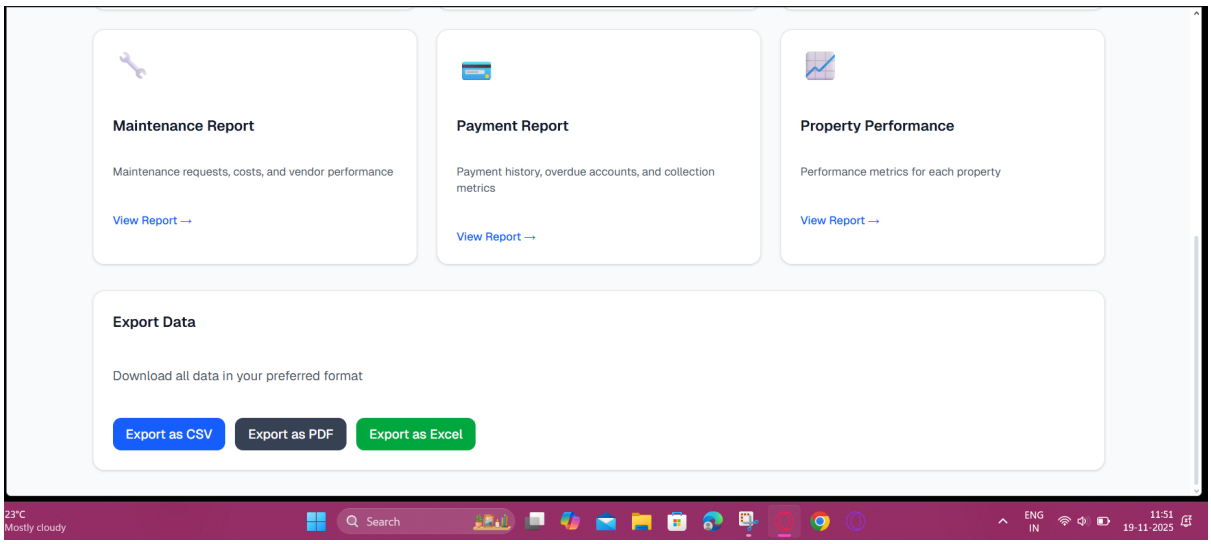

Figure 5.8: Reports Page

Chapter 6

Testing

The primary objective of testing is to ensure that the software meets its functional and non-functional requirements. Testing verifies correctness, reliability, performance, and usability of the system before deployment.

6.1 Types of Testing Performed

Various types of testing were performed to ensure comprehensive validation of the system. Each type addresses specific aspects of the software and helps identify potential defects.

6.1.1 Unit Testing

Unit testing was carried out to validate individual components and functions of the system. Examples include authentication logic, property creation functions, and database fetch operations. Each function was tested independently to ensure correct behavior.

6.1.2 Integration Testing

Integration testing ensured that modules interacted correctly with each other. This included verifying the connection between frontend components and the Supabase backend, as well as testing navigation flow between pages such as Login, Dashboard, and Property pages.

6.1.3 Functional Testing

Functional testing focused on validating that all features worked as expected. This included adding properties, editing property details, managing tenants, handling leases, and verifying authentication and authorization flows.

6.1.4 UI/UX Testing

UI/UX testing involved checking the visual layout, responsiveness, and user friendliness of the system. Elements such as buttons, forms, navigation menus, and cards were evaluated for consistency and ease of use across devices.

6.1.5 Performance Testing

Performance testing checked the speed and efficiency of the system. This included page load times, database query performance, and the behavior of the application when multiple properties or tenants were loaded.

Chapter 7

Conclusion and Future Scope

Conclusion

The Property Management System developed using Next.js, Supabase, and modern UI components successfully achieves its core objective of simplifying property-related tasks for owners, tenants, and administrators. The system provides an efficient platform for managing property details, tenant information, leases, payments, and overall records in a structured digital format.

Throughout the development process, multiple levels of testing—including unit, integration, functional, and UI/UX testing—ensured that the system is reliable, secure, and user-friendly. The application offers an intuitive interface, fast performance, and smooth navigation, making property management easier and more accessible.

Overall, the project serves as a complete digital solution that minimizes manual work, reduces errors, and enhances productivity for users involved in property management.

Future Scope

Although the system meets its initial goals, there are several opportunities for enhancement and expansion in future iterations:

- **Mobile Application:** A dedicated Android and iOS app can be developed to provide quicker access and real-time updates.
- **Automated Payment Gateway:** Integration with online payment systems such as Razorpay or Stripe can automate rent collection.
- **Analytics Dashboard:** Advanced analytics can be added to show financial statistics, rent trends, vacancy rates, and property performance.

- **Maintenance Request System:** Tenants can raise maintenance tickets that owners or managers can track and resolve.
- **Notification System:** SMS/email app notifications for rent reminders, lease renewal dates, and payment confirmations.
- **Multi-User Roles:** Additional roles such as property managers, accountants, or agents can be added with different permission levels.
- **AI-Based Recommendations:** Implement AI features to predict rent pricing, detect anomalies, or recommend property improvements.
- **Document Upload Storage:** Allow users to upload lease documents, ID proofs, and payment receipts directly into the system.

These additions would significantly enhance functionality, user experience, and scalability, making the system suitable for professional or commercial use.

References

- S. Pressman, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.
- Ian Sommerville, *Software Engineering*, Pearson Education.
- Official React Documentation: https://react.dev
- Vercel Deployment Documentation: https://vercel.com/docs
- Supabase Documentation: https://supabase.com/docs
- MDN Web Docs: https://developer.mozilla.org